



Efficient Training and Inference

Large Language Models and Foundation Models

EE269 Final Presentation

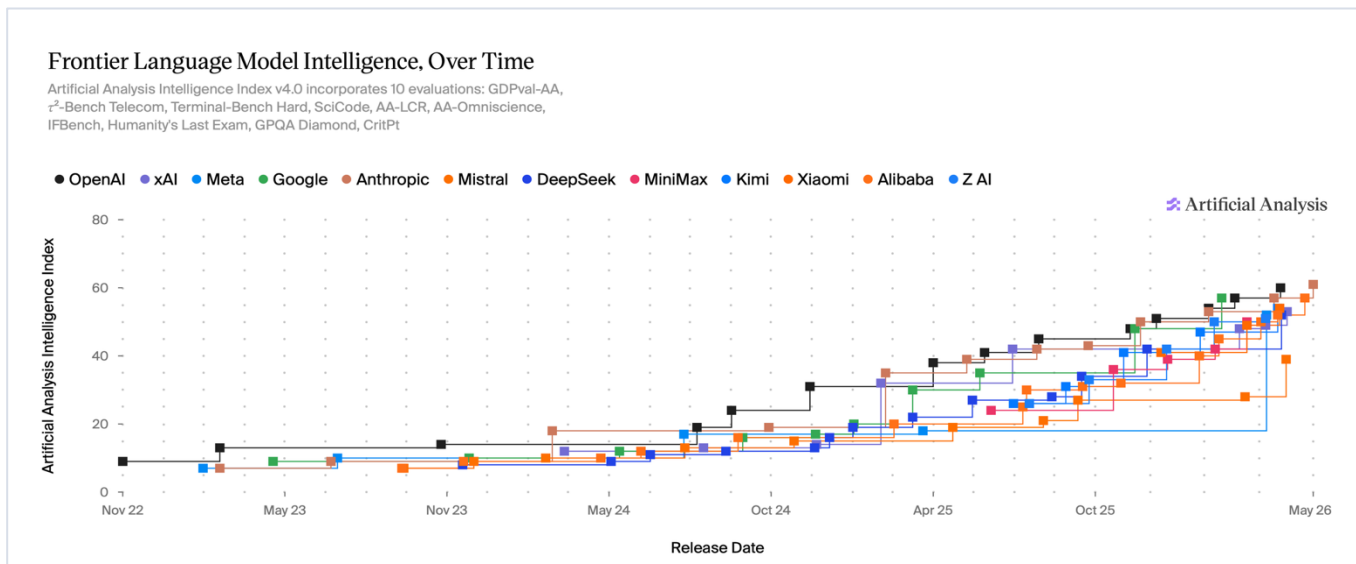
June 2, 2026

Zeli Liu | Weilun Wang | Zifan Li | Zirui Lin

- 01 **Efficient training**
- 02 **Model compression**
- 03 **Model-level inference**
- 04 **System-level serving**

Why Efficiency Matters

Scaling brings capability, but resource cost determines deployability



Capability must be evaluated together with memory, latency, throughput, and energy.

Presentation Roadmap

Each section focuses on a different layer of efficiency

Presenter	Section
Zeli Liu	Efficient Training
Weilun Wang	Model Compression
Zifan Li	Efficient Architecture + Model-Level Inference
Zirui Lin	System-Level Inference + Frameworks

Efficient Training

From pre-training efficiency to parameter-efficient adaptation

Topics

- Efficient pre-training
- Distributed training
- Parameter-efficient fine-tuning
- Memory-efficient fine-tuning

Training bottlenecks

- computing cost
- activation memory
- optimizer states
- gradients and parameters
- communication overhead

Key idea

Efficient training combines numerical, algorithmic, and distributed-system choices rather than relying on one isolated trick.

The objective is to make large-scale training and adaptation feasible under realistic memory and compute budgets.

Efficient Pre-Training Methods

Reduce compute, memory, wall-clock time, and communication overhead

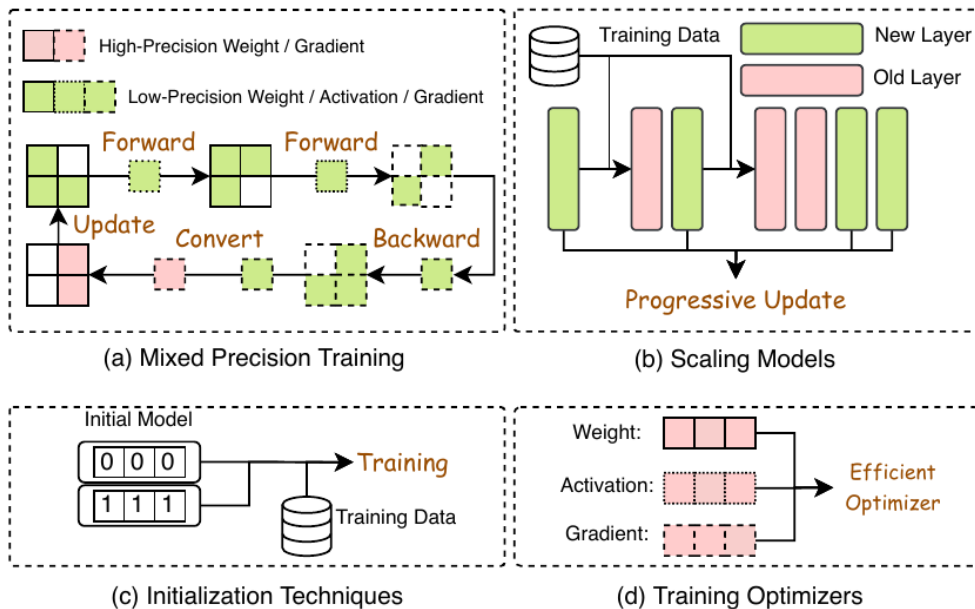
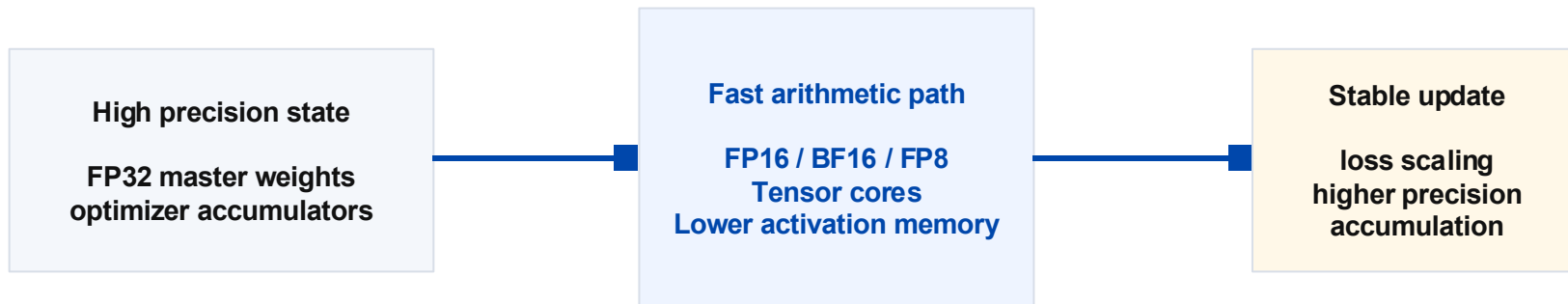


Figure 7, Wan et al. (2024): efficient pre-training techniques for LLMs.

Mixed Precision Training

Use low precision for throughput while protecting sensitive updates



Why BF16 is common

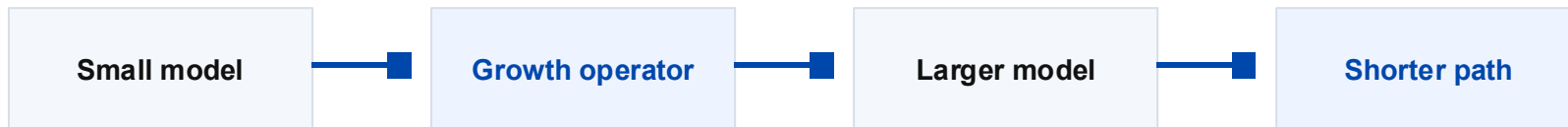
BF16 preserves a wider exponent range than FP16, making large-scale training more stable.

Co-design view

Precision choices depend on model sensitivity, numerical stability, and hardware support.

Model Growth, Initialization, and Optimizers

A cheaper route to scale can reuse computation and improve convergence



Growth examples

- progressive stacking
- CompoundGrow
- bert2BERT
- Knowledge Inheritance
- LiGO and Mango

Stability

- DeepNet-style initialization
- residual scaling
- normalization choices

Optimizers

- Lion reduces state
- Sophia uses a lightweight second-order estimate
- objective: faster convergence per token

Distributed Training: ZeRO and FSDP

Sharding reduces memory, but communication becomes the new constraint

Method	Sharded states	Memory benefit	System trade-off
ZeRO-1	optimizer states	moderate	low communication overhead
ZeRO-2	optimizer + gradients	higher	common multi-node choice
ZeRO-3	optimizer + gradients + parameters	highest	more communication
FSDP	full parameter/state sharding	high	hybrid strategies possible

Offloading extends capacity by moving states to CPU or NVMe, but data movement can dominate.

Parameter-Efficient Fine-Tuning

Freeze the backbone and train a small task-specific update

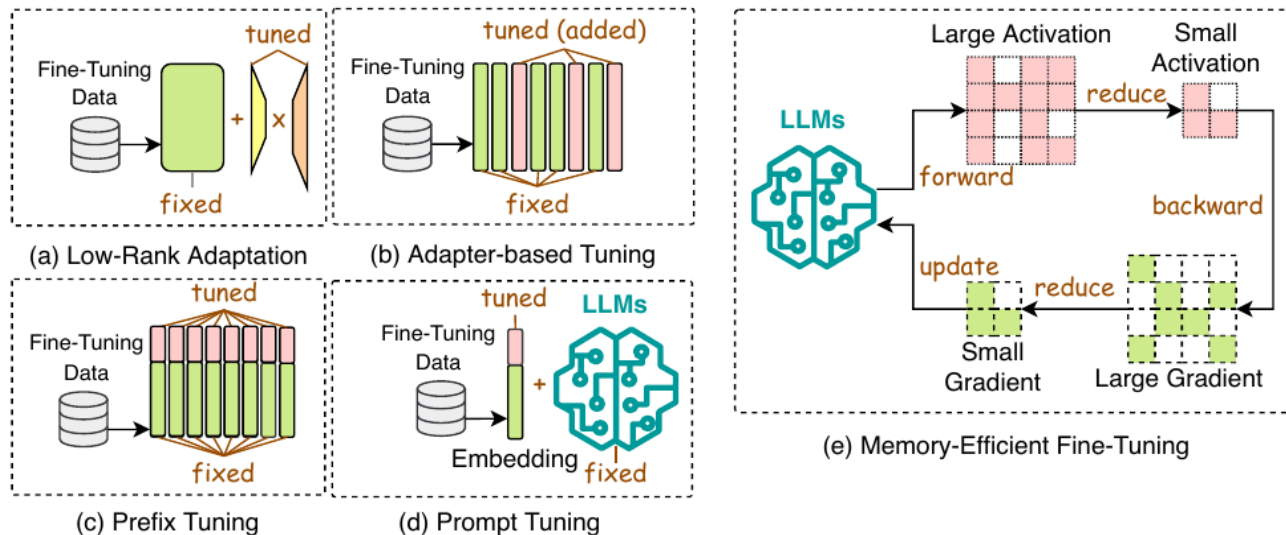


Figure 9, Wan et al. (2024): parameter-efficient and memory-efficient fine-tuning.

Memory-Efficient Fine-Tuning

Target the real memory bottlenecks: activations, gradients, and optimizer states

QLoRA

- 4-bit base model
- LoRA adapters
- full adaptation with much lower GPU memory

LOMO

- combines gradient computation and update
- removes optimizer state storage
- reduces gradient memory

MeZO

- zeroth-order gradient estimate
- two forward passes
- memory close to inference

**Insight: Efficient training lowers production and adaptation cost;
Deployment still requires model compression and inference optimization.**

Compression

Why compression?

1 Scale

GPT-3 has 175B parameters. LLaMA-2 70B requires 71,680 GPU-days to train and 80+ GB VRAM to load — more than a single A100.

2 Inference Cost

Larger models = fewer tokens/sec. KV-cache grows with sequence length, causing OOM errors and latency spikes at scale.

3 Training Expense

BLOOM 176B: 384 GPUs × 105 days. LLaMA 65B: 2,048 GPUs × 21 days. Training GPT-3 costs ~\$4–12M in compute alone.

4 Access & Deployment

Without compression, only big labs can run frontier models. A quantized 7B fits on a laptop; the same model at FP16 requires a server.

Compression classification

- Quantization
 - Post-Training Quantization: Weight-Only Quantization, Weight-Activation Co-Quantization
 - Quantization-Aware Training
- Parameter Pruning
 - Structured Pruning
 - Unstructured Pruning
- Low-Rank Approximation
- Knowledge Distillation
 - White-Box KD
 - White-Box KD

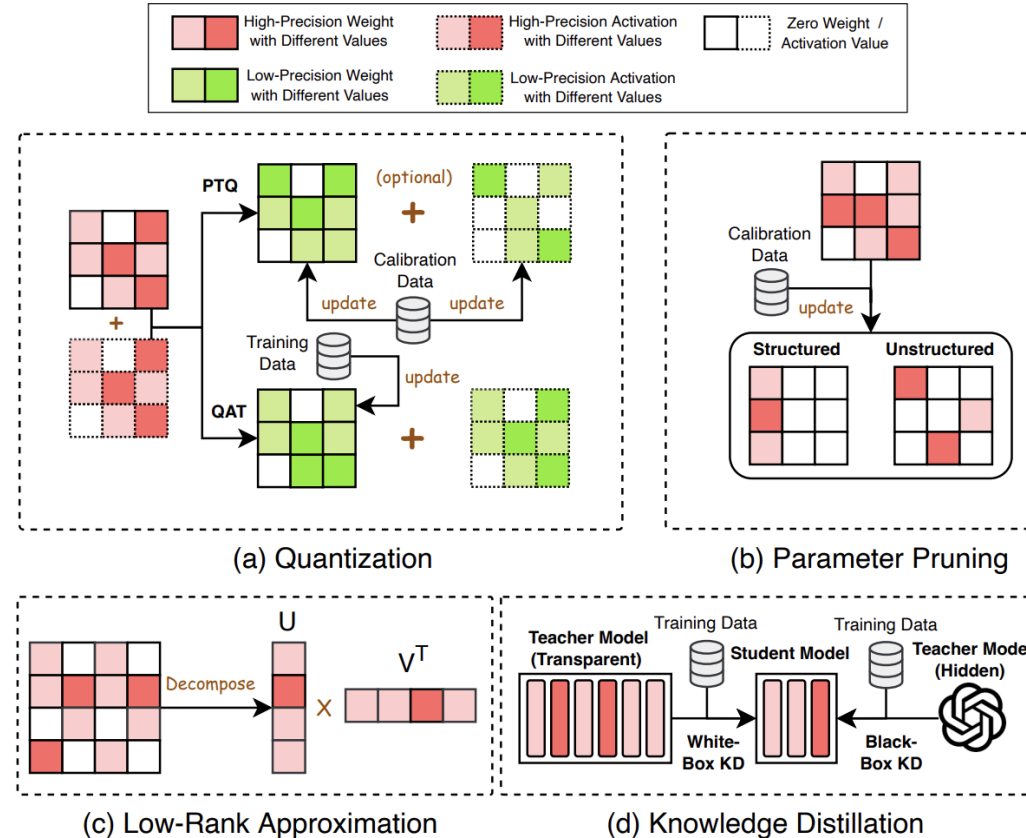


Figure 5: Illustrations of model compression techniques for LLMs.

Quantization: PTQ vs QAT

$$X_L = \text{Round}\left(\frac{X_H}{\Delta}\right), \quad \Delta = \frac{\max(X_H) - \min(X_H)}{2^b - 1}$$

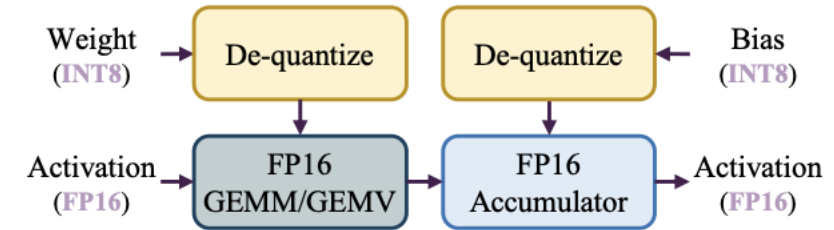
PTQ:

Weight-Only PTQ

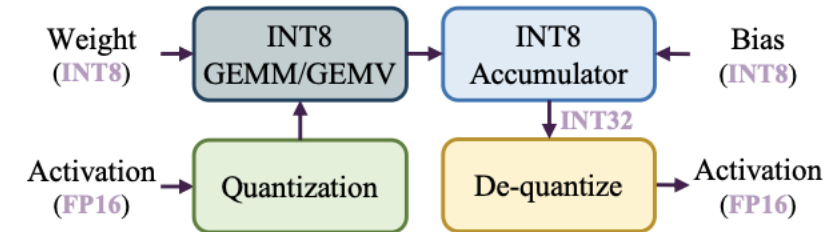
Weight-Activation Co-Quantization

QAT:

$$\min_{\theta} \mathbb{E}_{(x,y)} [\mathcal{L}(f_{\hat{\theta}}(x), y)]$$



(a) Weight-only Quantization



(b) Weight-Activation Quantization

Fig. 9. (a) The inference workflow of Weight-only Quantization. (b) The inference workflow of Weight-Activation Quantization.

Quantization: granularity & KV/activation

granularity:

Per-tensor

$$\Delta = \frac{\text{absmax}(W)}{2^{b-1} - 1}$$

Per-channel

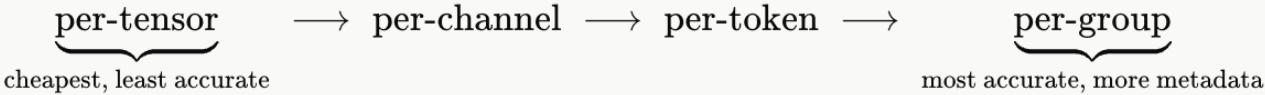
$$\Delta_i = \frac{\text{absmax}(W_{i,:})}{2^{b-1} - 1}, \quad i = 1, \dots, m$$

Per-token

$$\Delta_t = \frac{\text{absmax}(X_{t,:})}{2^{b-1} - 1}, \quad t = 1, \dots, T$$

Group-wise

$$\Delta_{i,k} = \frac{\text{absmax}(W_{i,kg:(k+1)g})}{2^{b-1} - 1}$$



KV cache/activation(Hardware friendly)

$$\text{KV size} = 2 \times L \times H \times d \times T \times \text{bytes}$$

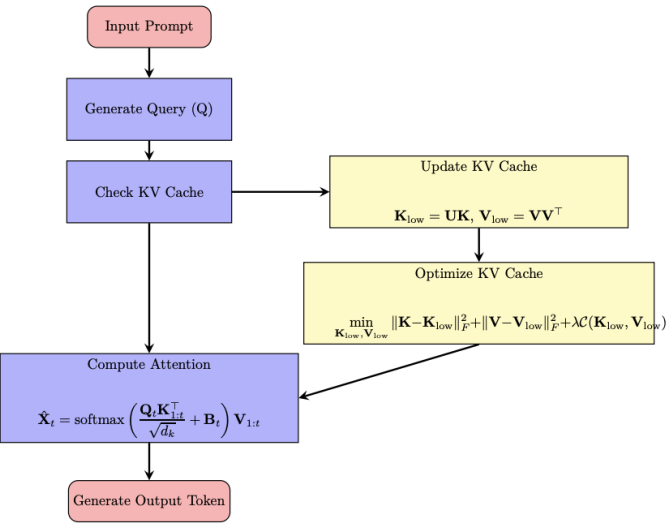


Figure 2: KV Cache Design Pipeline

Pruning: structured vs unstructured

Structured:

Attention heads

Intermediate (FFN) neurons

Layers

Hidden dimensions

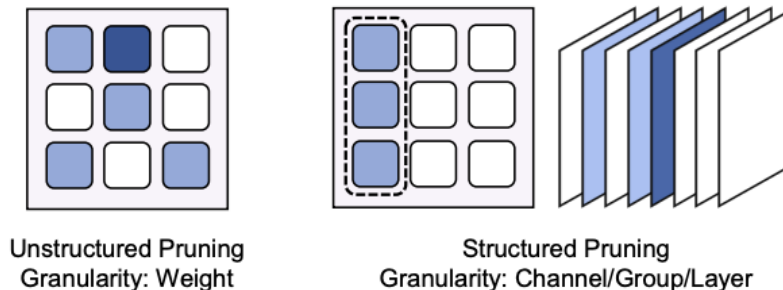


Fig. 10. Illustration of Unstructured Pruning (left) and Structured Pruning (right).

$$I(g) = \left| \sum_{w \in g} \frac{\partial \mathcal{L}}{\partial w} \cdot w \right|$$

Unstructured

$$\arg \min_{M_\ell, \hat{W}_\ell} \|W_\ell X_\ell - (\hat{W}_\ell \odot M_\ell) X_\ell\|_2^2.$$

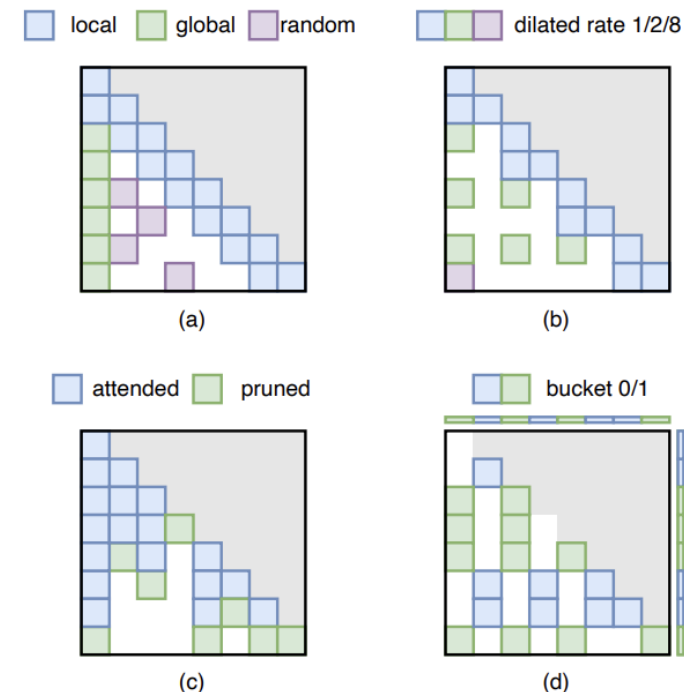


Fig. 11. Examples of different sparse attention masks. (a) Static mask with local, global, and random attention pattern. (b) Static mask with dilated attention pattern of different dilated rate. (c) Dynamic token pruning. (d) Dynamic attention pruning.

Distillation: teacher– student

$$p_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

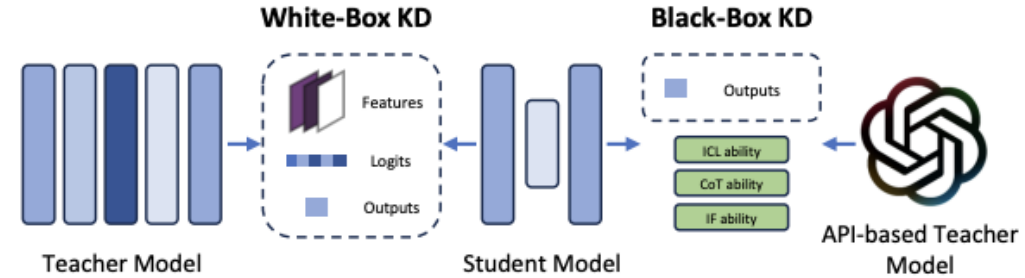


Fig. 12. Illustration of White-Box KD (left) and Black-Box KD (right).

soft distillation:

$$\mathcal{L}_{\text{distill}} = \lambda_{\text{fwd}} \sum_{i=1}^N \alpha_i \cdot D_{\text{KL}}(p_{\text{teacher},i} \| p_{\text{student},i}) + \lambda_{\text{rev}} \sum_{i=1}^N \beta_i \cdot D_{\text{KL}}(p_{\text{student},i} \| p_{\text{teacher},i}),$$

$$D_{\text{KL}}(p \| q) = \sum_j p_j \log \frac{p_j}{q_j}$$

Hard Distillation

$$\mathcal{L}_{\text{hard}} = - \sum_{i=1}^N \sum_{j=1}^C y_{i,j}^{\text{teacher}} \cdot \log p_{i,j}^{\text{student}},$$

Low-rank

Basic method(SVD):

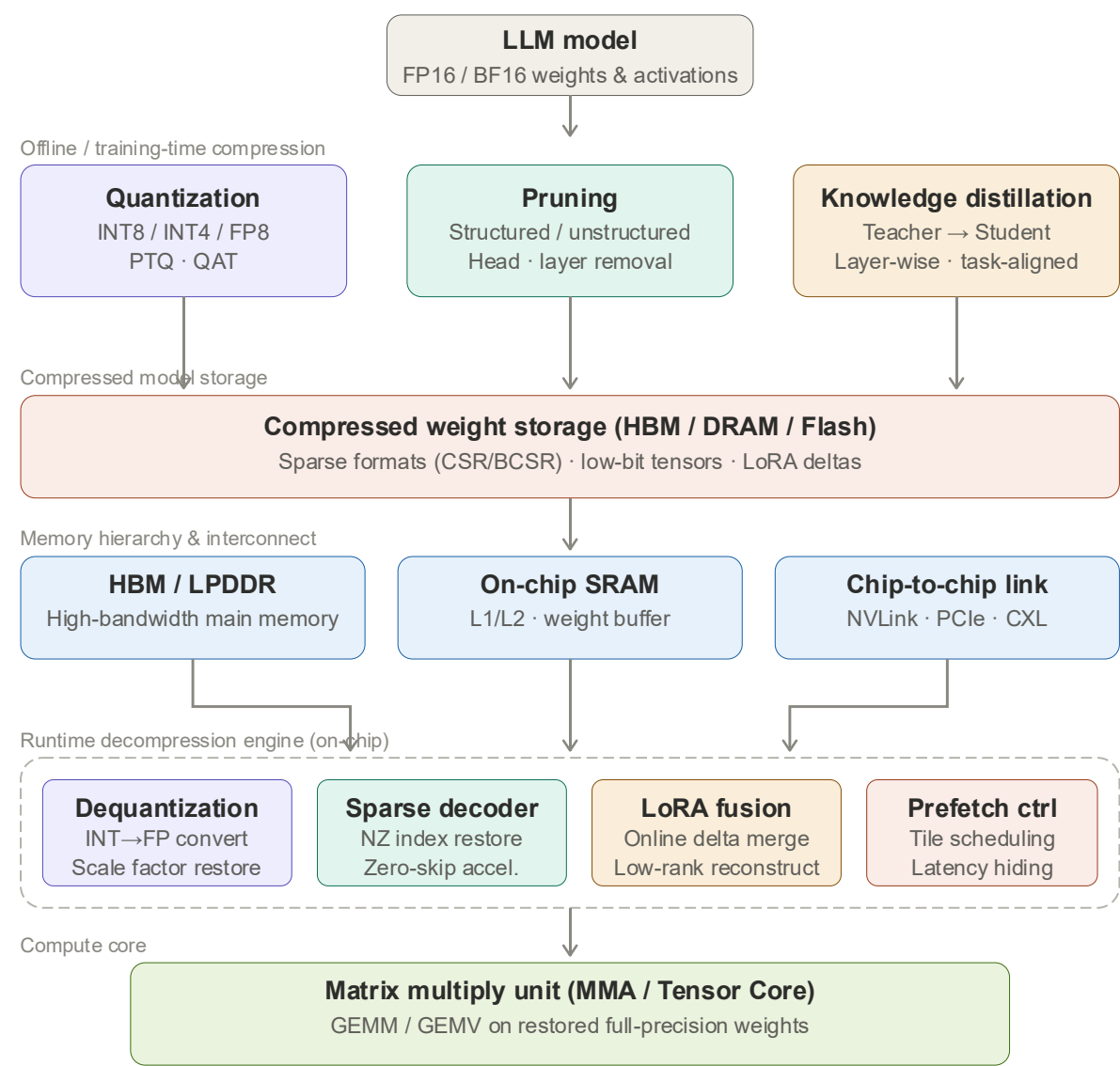
$\mathbf{U}$ and $\mathbf{V}$ such that $\mathbf{W} \approx \mathbf{U}\mathbf{V}^\top$, where $\mathbf{U} \in \mathbb{R}^{m \times r}$, $\mathbf{V} \in \mathbb{R}^{n \times r}$,

Ratio:

$$\frac{mn}{r(m+n)}$$

Hardware support

Without robust hardware support, many processes—such as decomposition and dequantization—could lead to catastrophic results.



Thank you

Efficient Architecture and Model-Level Inference

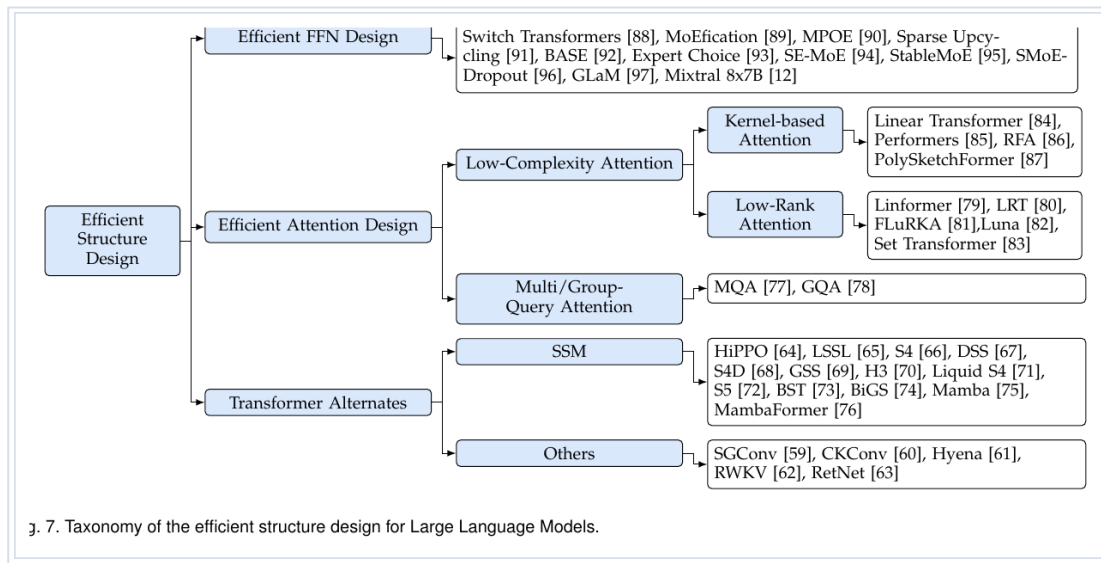
Reducing the cost of each generated token with model-side design

Section goal

- Lower latency per request
- Increase token throughput
- Reduce KV-cache and weight traffic
- Preserve output quality

Scope

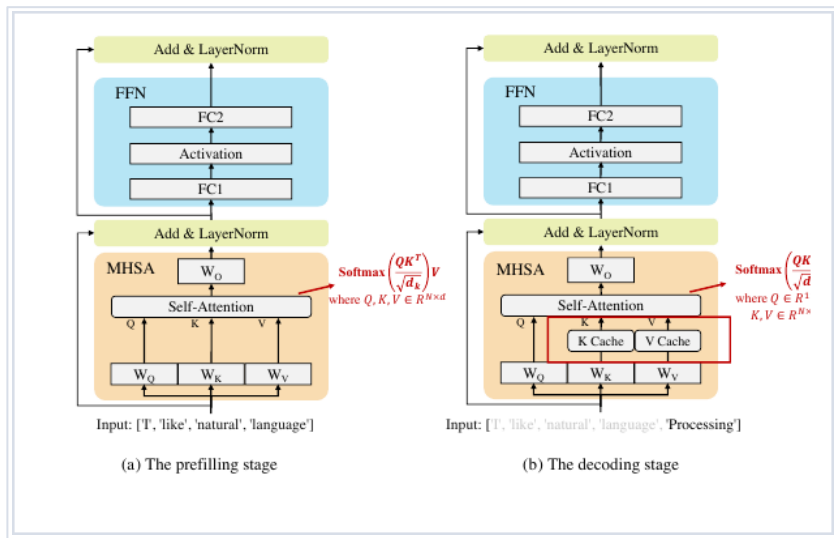
Model-side efficiency includes efficient structure design plus inference algorithms. Serving systems are the next layer.



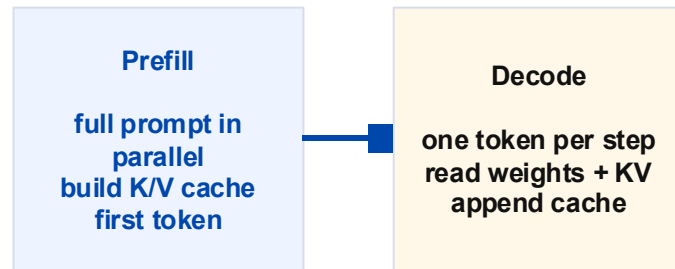
S2 Fig. 7: efficient structure design taxonomy

Inference Pipeline: Prefill vs. Decode

The two phases optimize for different metrics and bottlenecks



S2 Fig. 2: prefill vs. decode



Metric split

TTFT measures prefill responsiveness; TPOT measures decode speed after generation starts.

This split explains why kernel, cache, and batching optimizations often help different parts of a request.

Three Root Bottlenecks

Model size, quadratic attention, and autoregressive decoding drive cost

1. Model size

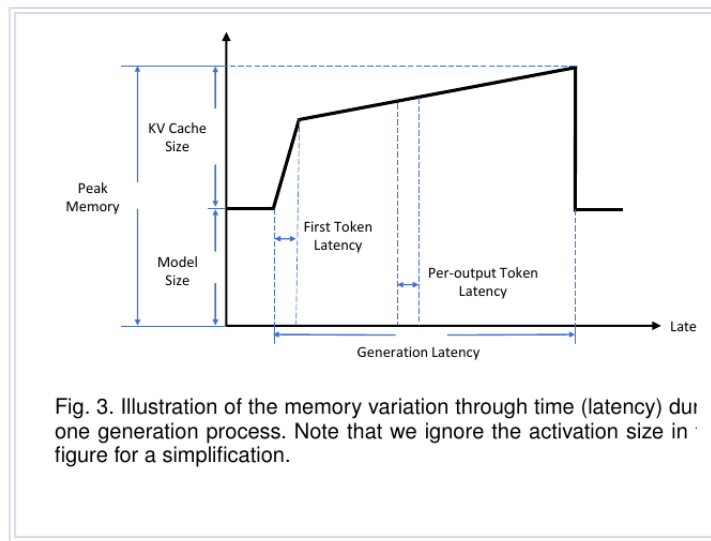
- weight memory
- activation memory
- KV-cache footprint
- lower batch capacity

2. Attention $O(n^2)$

- prefill scales with prompt length
- dense token-token interactions
- long-context pressure

3. Autoregressive decode

- one token at a time
- repeated weight access
- growing KV cache
- memory-bandwidth pressure



S2 Fig. 3: memory and latency during generation

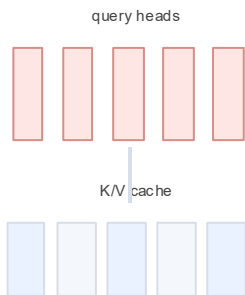
Model architecture attacks per-token work; inference algorithms attack cache traffic and target-model calls.

Efficient Attention: MHA to MQA and GQA

Share K/V states to reduce decode-time memory traffic

MHA

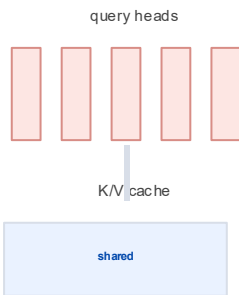
each head owns K/V



Less K/V duplication

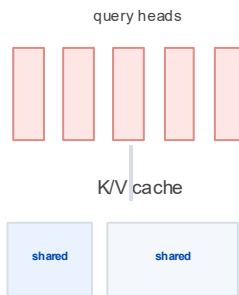
MQA

all heads share K/V

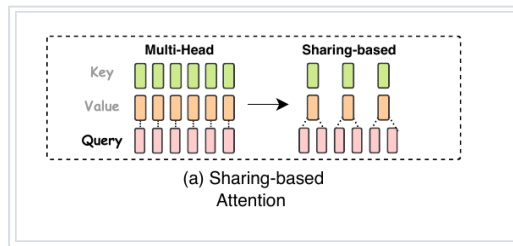


GQA

heads share within groups



lower cache footprint and
memory traffic during decode



sharing-based attention

MQA and GQA are sharing-based attention designs: queries stay multi-head, but K/V states are reused across heads or groups.

FlashAttention

Exact attention becomes faster by reducing memory movement

IO-aware attention

- tile Q, K, and V
- compute blocks on SRAM
- avoid materializing full attention matrix
- reduce HBM traffic

Where it helps

Most useful when attention is memory-traffic limited: long prompts, large batches, and prefill-heavy workloads.

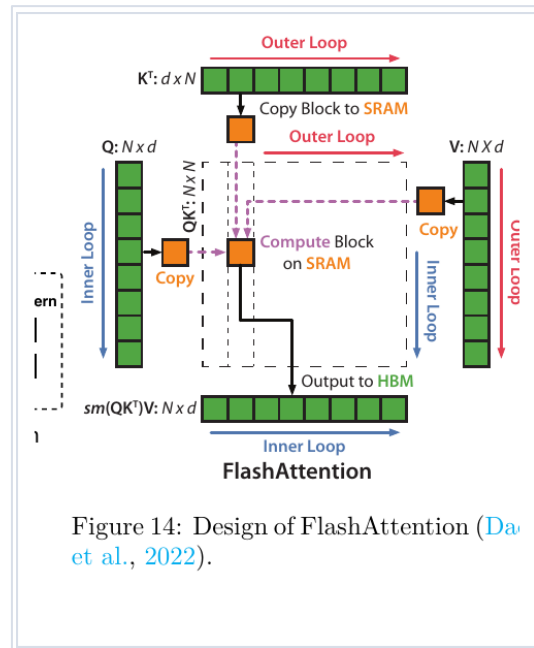
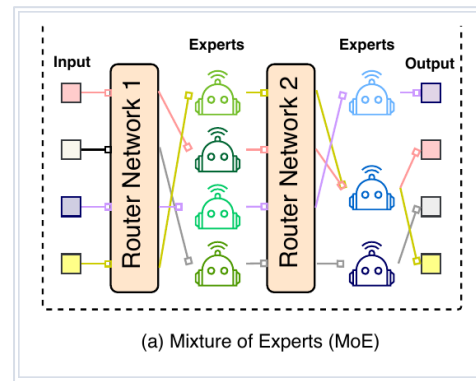
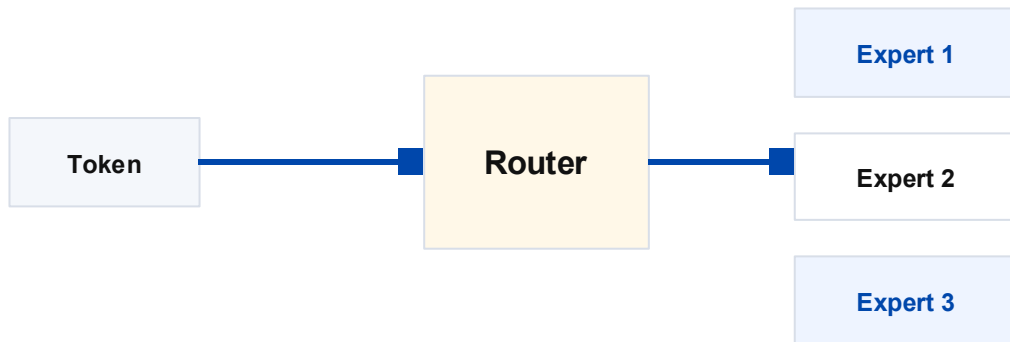


Figure 14: Design of FlashAttention (Dav et al., 2022).

Same attention result, less movement between HBM and on-chip memory.

Efficient FFN: Mixture-of-Experts

Sparse activation expands capacity without activating all parameters



Efficiency promise

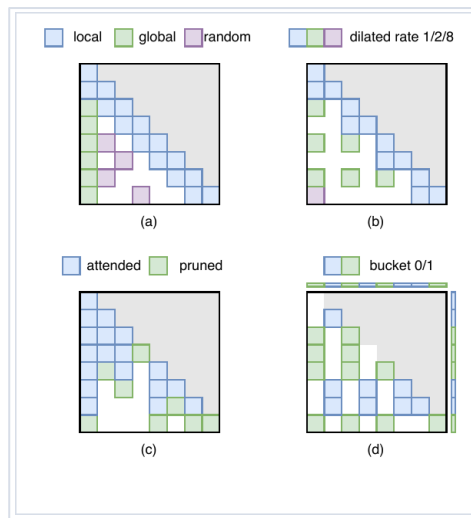
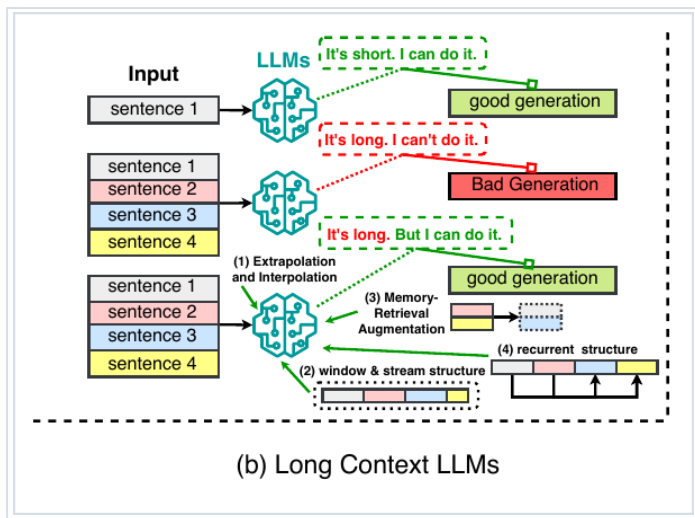
Only selected experts are activated, so capacity can grow faster than active FLOPs.

Main caveat

Routing imbalance and all-to-all communication can reduce the expected savings.

Long Context: Locality, Sparsity, and Alternatives

Avoid paying full attention over every token pair



Design options

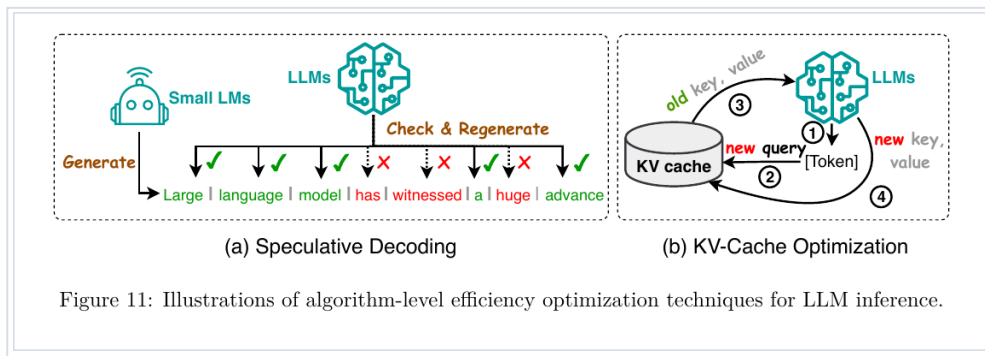
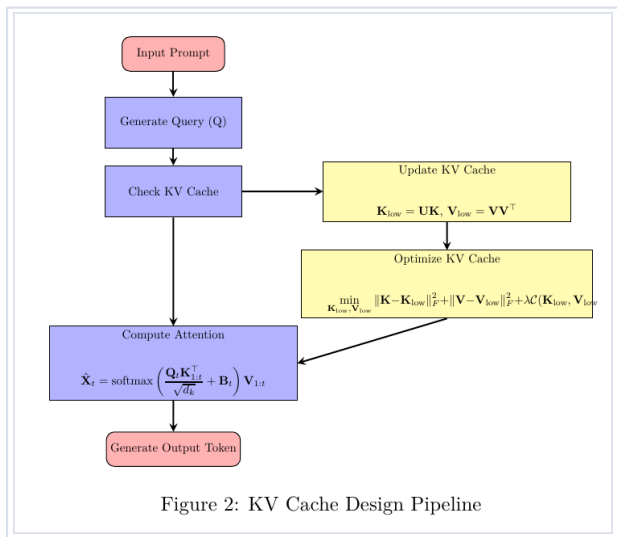
sliding window / sparse masks / SSM and Mamba-style alternatives

Shared objective

Reduce context-length sensitivity without discarding useful evidence.

KV Cache + Speculative Decoding

Reduce memory traffic or reduce expensive target-model calls



KV-cache optimization

- quantize K/V values
- evict low-utility tokens
- keep high-utility context

Speculative decoding

- draft model proposes tokens
- target model verifies in parallel
- accepted tokens amortize one call

System-Level Inference Optimization

From Efficient Models to Efficient Services

Why is System-Level Inference Optimization necessary?

- Model scale: Larger model size
- Long context: Longer input and output context
- Autoregressing Decoding
- Growing KV cache
- Asynchronous multi-user requests

What is System-Level Optimization?

1. Inference Engine

- graph / operator optimization
- speculative decoding
- offloading

2. Serving System

- memory management
- batching
- scheduling
- distributed serving

What metrics to be improved?

- TTFT: Time to First Token
- TPOT: Time Per Output Token
- Throughput
- Memory Footprint
- Energy / Cost

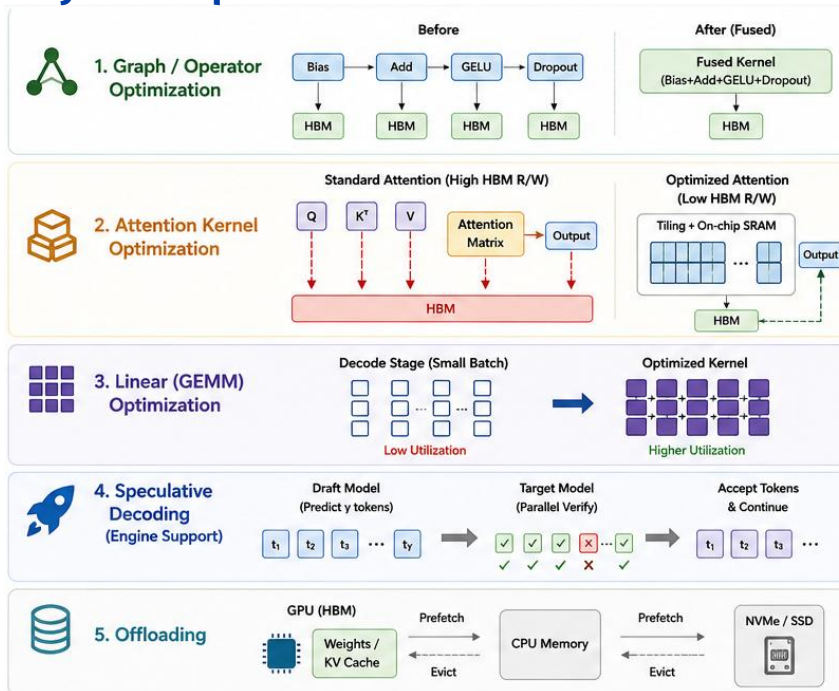
Inference Engine Optimization

Make Every Forward Pass Faster

What is Inference Engine?

- Focus on a single Forward pass
- Optimize the computational graph, operators and kernels
- Usually no change to model weights and semantics
- **Key challenge:** LLM decoding is memory-bound(HBM bandwidth is the bottleneck)

Key Techniques



Final Purpose

- Reduce memory movement
- Improve hardware efficiency
- Shorten token latency
- Increase inference throughput
- Lower deployment cost

Speculative Decoding

Reduce the Number of Expensive Decoding Steps

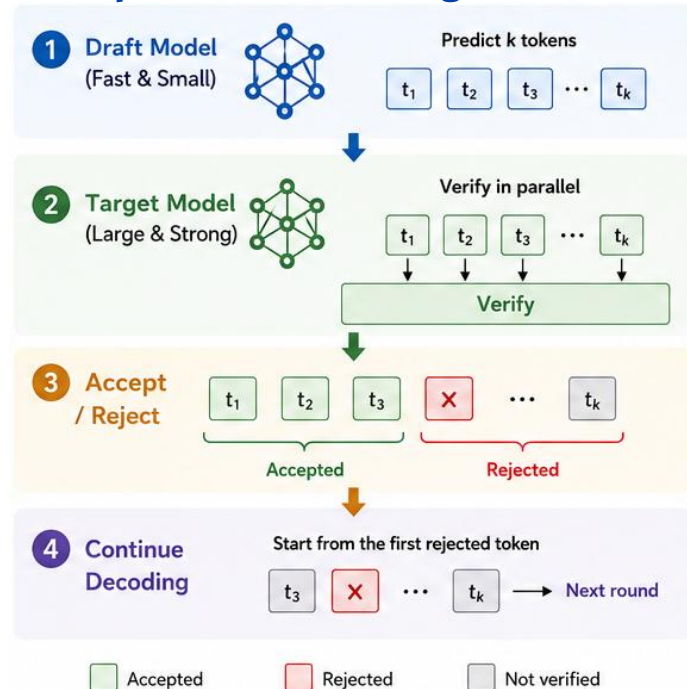
Why needed?

- Autoregressive decoding is sequential
- Large Model is underutilized
- High Decoding Cost:
 1. Many decoding steps;
 2. High latency per token.

Key Idea

Predict multiple tokens first, then verify them in parallel.

How Speculative Decoding works



What Speculative Decoding Achieves?

- Cheap draft model
- High acceptance rate
- Parallel verification
- Fewer target-model calls
- Quality preserved

Offloading

Run Larger Models with limited GPU Memory

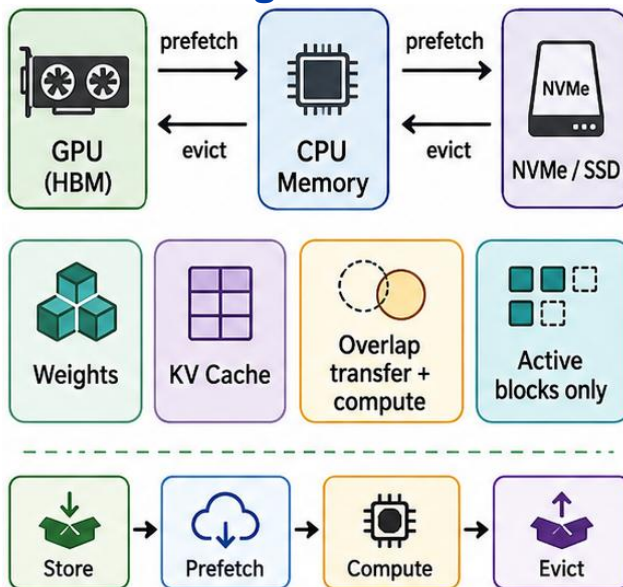
Why Offloading is necessary?

- Limited HBM
- Large model weights
- Growing KV cache
- Long context
- Memory bottleneck

Key Idea:

Offloading trades extra data movement for better memory capacity and deployability

How Offloading works?



System Cost Model

Memory Constraint:

$$M_{active} \leq C_{GPU}$$

KV Cache Growth:

$$M_{KV} = 2 \cdot L \cdot B \cdot S \cdot H_{kv} \cdot d_h \cdot \text{bytes}$$

Transfer Cost:

$$T_{move} = \frac{D_{offload}}{B_{link}}$$

Key Goal:

$$T_{step} \approx \max(T_{compute}, T_{move})$$

KV Cache & PagedAttention

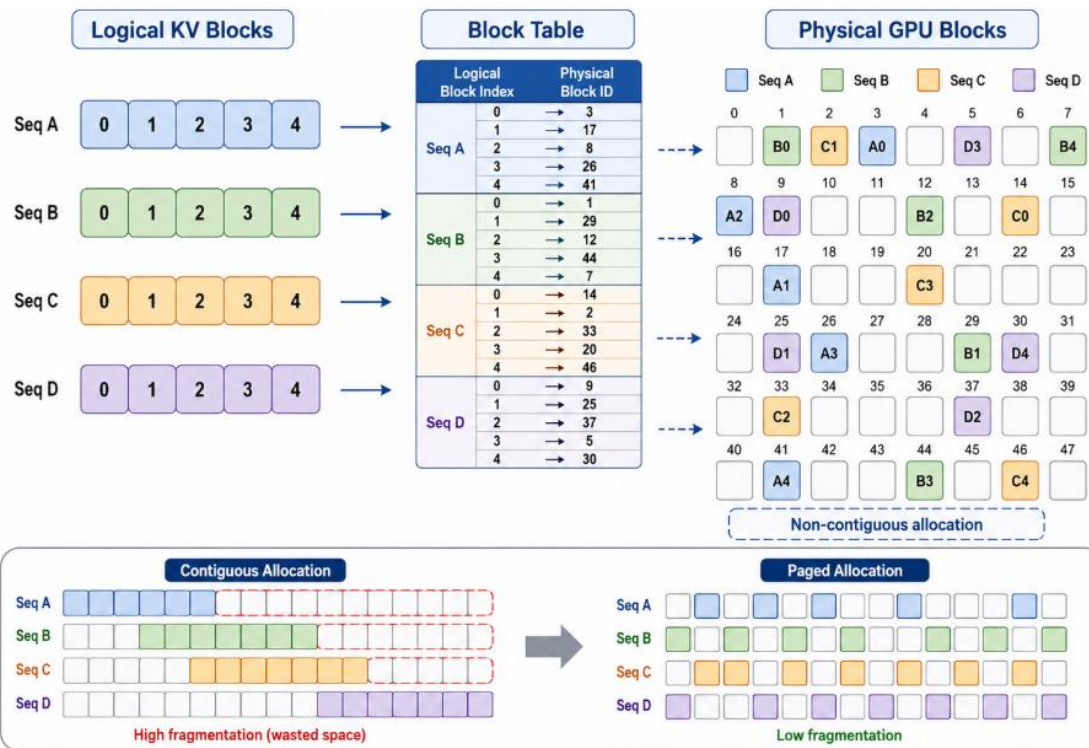
Improve Memory Utilization for LLM Serving

What is KV cache?

- Stores previous Keys and Values
- Reused during decoding
- Avoids recomputing history
- Grows with context length
- Major memory bottleneck

Why PagedAttention matters?

- Less fragmentation
- Higher memory utilization
- More concurrent requests
- Shared prefix reuse
- Higher serving throughput



Continuous Batching

Keep the GPU Busy During Online Serving

Problems with Static Batching

- Variable request length
- Short requests finish early
- Idle batch slots
- Low utilization
- Throughput loss

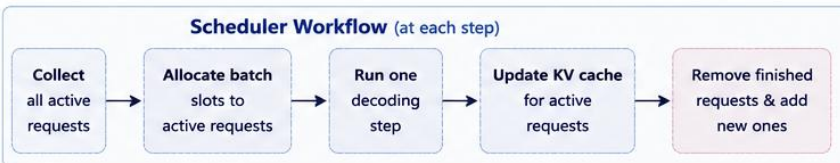
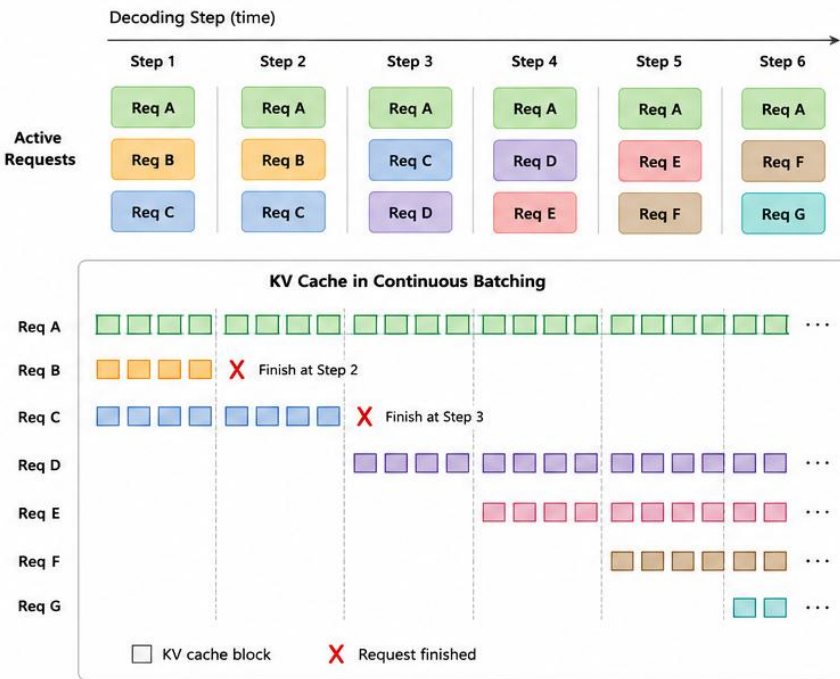
What is Continuous Batching?

- It is to keep the batch “full” of active requests.
- When a request finishes, add a new one immediately.



How Continuous Batching works?

At each decoding step, schedule all active requests into the batch.



Scheduling & P/D Disaggregation

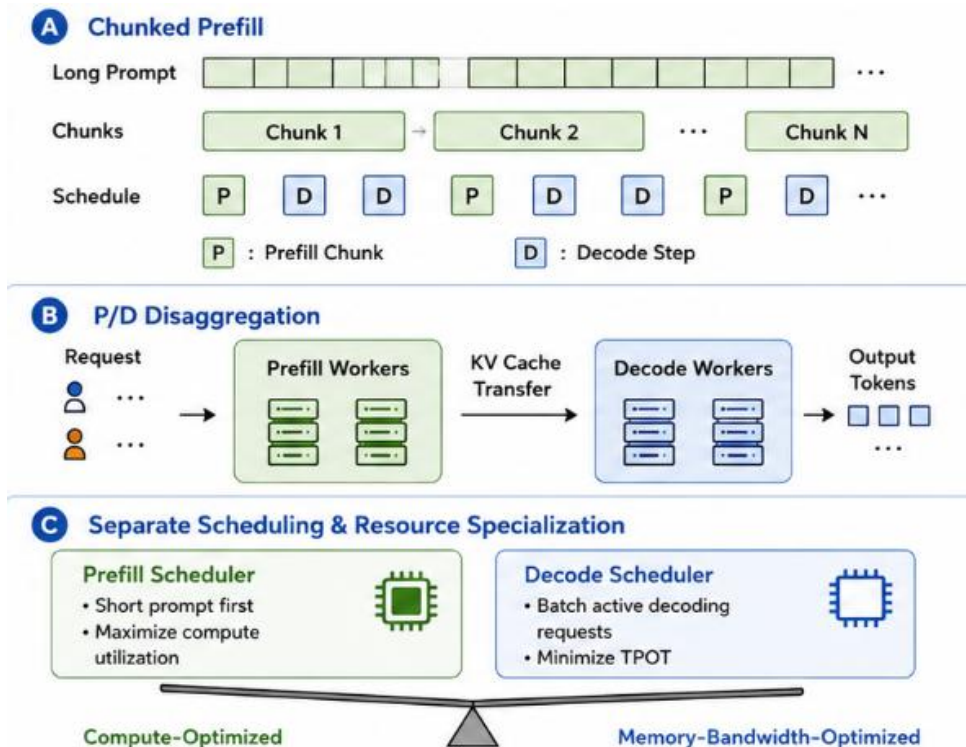
Seperate Predill and Decode for Better Serving

Scheduling

- Request ordering
- Worker assignment
- Decode batching
- TTFT / TPOT trade-off

P/D Disaggregation

- Prefill pool
- KV cache transfer
- Decode pool
- Resource specialization



Framework Comparison

Different Systems for Different Deployment Goals

Model	Inference Optimization				Inference (tokens/s)	Serving Optimization			Serving (req/s)
	Attention	Linear	Graph	Speculative Decoding		Memory	Batching	Scheduling	
HuggingFace [260]				✓	38.963	-	-	-	-
DeepSpeed [258]	✓	✓			80.947	blocked paged	split-and-fuse continuous batching	decode prioritized prefill prioritized	6.78
vLLM [51]	✓			✓	90.052	paged	continuous batching	prefill prioritized	7.11
OpenPPL [263]	✓		✓		81.169	-	-	-	-
FlashDecoding++ [253]	✓	✓	✓	✓	106.636	-	-	-	-
LightLLM [278]	✓				73.599	token-wise paged	split-and-fuse continuous batching	prefill prioritized	10.29
TensorRT-LLM [222]	✓	✓	✓	✓	92.512	token-wise paged	continuous batching	-	5.87

Observations

1. Frameworks optimize different layers
2. vLLM is serving-oriented
3. FlashDecoding++ is engine-oriented
4. DeepSpeed and LightLLM emphasize scheduling

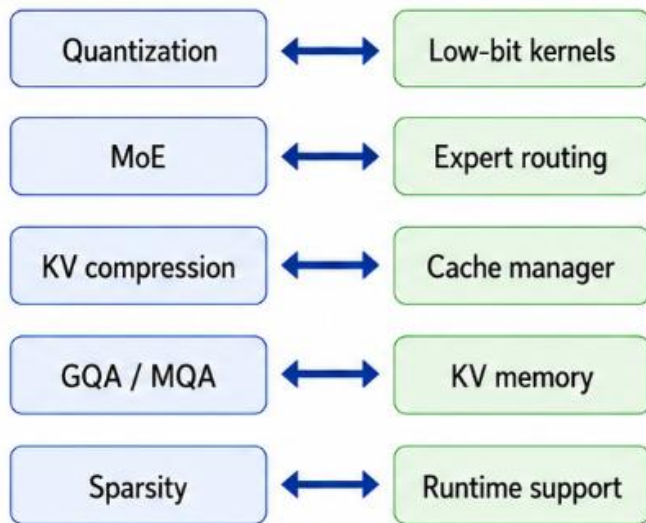
Model-System Co-Design

Efficiency Requires Coordination Across the Stack

Why Co-Design important

- Model choices affect serving.
- System constraints affect model design.
- Single-Layer optimization is limited.
- Deployment-aware design is needed.
- Full-stack efficiency

Key example



Design Principles

- Design for hardware
- Optimize memory movement
- Match structure to runtime
- Expose structure to scheduler
- Evaluate end-to-end serving

Challenges & Directions

Future Directions for Efficient LLM Inference

Key Challenges

- **Memory Bottleneck:**
KV cache and model states still dominate memory usage.
- **Long-context Serving:**
Efficiency drops with very long sequences.
- **Scheduling Complexity:**
Balancing TTFT, TPOT, throughput, and fairness is hard.
- **Heterogeneous Hardware:**
Portability and full utilization remain difficult.
- **Co-design Gap:**
Model advances need better system and hardware support.

Future Directions

- **Smarter Scheduling:**
Learn to predict and adapt to request patterns.
- **Better Memory Management:**
Compression, offloading, and cache optimization.
- **End-to-end Co-design:**
Joint optimization across model, system, and hardware.
- **Specialized Hardware:**
More support for sparsity, low-bit, and large-context inference.
- **Standardized Benchmarking:**
Unified metrics for fair comparison.



Marlan and Rosemary Bourns
College of Engineering

THANK YOU